AI AGENT EXECUTION PLAYBOOK

Multi-Venture Retail And Cafe Phase Prompts

Twelve standalone, security-gated prompts for extending the existing local-first Retail POS into a shared Retail and Cafe platform.

```
Business Group > Venture/Company > Branch
```

Cafe Partner: Cafe-only administration

Customer: QR menu and table ordering

Staff: direct ordering, preparation, billing, and closing

Final Super Admin: Retail, Cafe, consolidated reporting, and governance

Use one phase per agent task. Do not advance until its security, migration, regression, and acceptance gates pass.

Version 1.0 | 11 August 2026

Contents

The page numbers below are generated from the final rendered document. PDF bookmarks are also included for each phase.

How To Use This Prompt Book

1. Run exactly one phase prompt at a time in the existing project workspace.
2. Do not start the next phase until the current phase report shows that all security and acceptance gates pass.
3. Give the agent access to the repository and all referenced Markdown documents.
4. Keep a verified backup before the first schema migration and before any destructive governance test.
5. Use a disposable database for seed resets, migration rehearsal, purge tests, and restore tests.
6. Do not paste all phase prompts into one task. The isolation and verification gates depend on sequential execution.

The prompts are deliberately self-contained. Each prompt tells the agent what to read, what to implement, what not to implement, what to test, and how to report completion.

Source-Of-Truth Order

Future agents must apply these documents in this order:

1. `PRD.md`
2. `PRD_HITECH_COMPETITIVE_ADDONS.md`
3. `PRD_MULTI_VENTURE_CAFE_EXPANSION.md`
4. `TRD_MULTI_VENTURE_CAFE_EXPANSION.md`
5. `docs/MULTI_VENTURE_CAFE_IMPLEMENTATION_PHASES.md`
6. `AGENT_STEP_BY_STEP_PROMPTS_MULTI_VENTURE_CAFE.md`

The original PRDs remain the contracts for Retail, inventory, POS, billing, GST readiness, ledgers, analytics, forecasting, AI, exports, and local-first deployment. The new PRD and TRD control multi-venture and Cafe-specific behavior.

Non-Negotiable Rules For Every Phase

- Preserve one FastAPI backend, one React application, and one local PostgreSQL database.
- Never expose PostgreSQL directly to the internet.
- Backend authorization must enforce business group, company/venture, branch, role, object, and action scope.
- A normal Admin is company-scoped. Only Final Super Admin is cross-venture.
- The Cafe Partner Admin must never receive Retail records, counts, names, routes, exports, AI results, or identifying error details.
- Never trust client-supplied company, branch, role, price, discount, tax, total, payment status, or stock effect.
- Customer QR and authenticated staff orders must use one Cafe order service.
- Reuse the existing invoice, payment, stock movement, sales compatibility, ledger, audit, and reporting foundations.
- Every stock change must create a stock movement.
- Financial and stock effects must be transactional and idempotent.
- Non-GST is the default until valid registration settings and controlled activation exist.
- Issued financial records use void/reversal. Exceptional purge must never be a silent delete.
- Preserve unrelated worktree changes.
- Run targeted tests and full relevant regression before reporting completion.

Phase 0 Prompt: Baseline, Scope Lock, And Recovery Safeguards

COPY-PASTE PROMPT

Copy and run this prompt exactly as one agent task:

You are preparing the existing AI-Powered Hybrid Retail Billing and POS project for its secure multi-venture Retail and Cafe expansion.

Before changing anything, read these files completely:

- PRD.md
- EXECUTION_FLOW_ANALYSIS.md
- PRD_HITECH_COMPETITIVE_ADDONS.md
- docs/ADDON_ARCHITECTURE_DECISIONS.md
- PRD_MULTI_VENTURE_CAFE_EXPANSION.md
- TRD_MULTI_VENTURE_CAFE_EXPANSION.md
- docs/MULTI_VENTURE_CAFE_IMPLEMENTATION_PHASES.md
- AGENT_STEP_BY_STEP_PROMPTS_MULTI_VENTURE_CAFE.md
- README.md

Then inspect the current code, models, services, routes, schemas, Alembic migrations, tests, seed data, frontend routing, environment templates, and git status.

Implement Phase 0 only: Baseline, Scope Lock, And Recovery Safeguards.

Goal:

Create a trustworthy pre-expansion baseline without changing application behavior, database schema, or business logic.

Required work:

1. Record the repository structure and current implementation status.
2. Record the current Alembic head and migration chain.
3. Run the complete backend test suite with the existing development/test environment.
4. Run frontend type checking and production build.
5. Inspect current roles, branch-scope behavior, Company assumptions, invoice/POS state, and known unfinished add-on features.
6. Confirm which database is being used for tests and which database URL is configured for local development.
7. Do not print or commit passwords, tokens, database secrets, or real customer data.
8. If a real local PostgreSQL database contains useful data, create a backup using the existing documented backup process. Never reset or restore over the working database.
9. If PostgreSQL is available, prepare or document a separate disposable database for migration rehearsal. Do not require a paid cloud database.
10. Create docs/MULTI_VENTURE_BASELINE.md containing:

- date and environment;
- current Alembic head;
- backend test count and result;

- frontend typecheck/build result;
- current demo users and role behavior without passwords beyond already documented development credentials;
- current Company/Branch/User assumptions;
- current invoice print/PDF readiness;
- current PostgreSQL connection blockers, if any;
- backup status;
- known failures and risks;
- migration prerequisites for Phase 1.

11. Add a traceability table mapping each MV-FR requirement range to the phase that will implement it.
12. Update documentation links only if needed. Clearly mark the Cafe expansion as planned, not implemented.

Security checks:

- PostgreSQL is not publicly exposed.
- .env and secrets remain ignored by git.
- No raw tokens, QR secrets, password hashes, or customer PII are written to docs/log output.
- Test and backup commands target the intended paths and databases.

Do not:

- Add or change database tables.
- Add Cafe routes or pages.
- Change role behavior.
- Change seed business data.
- Reset a real database.
- Implement later phases.

Verification commands when the environment supports them:

Backend:

```
cd backend
.\.venv\Scripts\python.exe -m pytest -q
alembic current
alembic heads
```

Frontend:

```
cd frontend
npm run typecheck
npm run build
```

Manual verification:

- Login as each current demo role.
- Confirm Retail POS, inventory, invoices, dashboards, purchase orders, forecasting, AI, and exports behave as before.
- Confirm a backup can be created without exposing secrets.

Acceptance criteria:

- docs/MULTI_VENTURE_BASELINE.md exists and is accurate.
- Existing test/build status is recorded.
- Migration and backup prerequisites are known.

- No runtime behavior or schema changed.

Report using this exact structure:

Phase completed:

Files changed:

Requirements mapped:

Commands run:

Verification results:

Backup/recovery status:

Known gaps or blockers:

Next recommended phase:

Phase 1 Prompt: Ownership, Venture, And Data-Scope Schema

COPY-PASTE PROMPT

Copy and run this prompt only after Phase 0 passes:

You are implementing Phase 1 of the approved multi-venture Retail and Cafe expansion.

Read completely before editing:

- PRD.md
- PRD_HITECH_COMPETITIVE_ADDONS.md
- PRD_MULTI_VENTURE_CAFE_EXPANSION.md
- TRD_MULTI_VENTURE_CAFE_EXPANSION.md, especially Sections 3, 5, and 6
- docs/MULTI_VENTURE_CAFE_IMPLEMENTATION_PHASES.md, Phase 1
- docs/MULTI_VENTURE_BASELINE.md
- current models, services, migrations, seed script, tests, and git status

Implement Phase 1 only: Ownership, Venture, And Data-Scope Schema.

Goal:

Create the BusinessGroup -> Company/Venture -> Branch hierarchy and safely backfill all existing business data into the Retail venture without changing financial totals or user-visible Retail behavior.

Architecture contract:

- Existing companies represent venture workspaces.
- Add one lightweight business-group/partnership parent.
- Existing data belongs to the Retail company.
- A minimal Cafe company and branch may be seeded after reconciliation.
- Existing global Admin becomes Super Admin during migration.
- Other existing users become Retail-company users.
- Cafe features and public routes are not built in this phase.

Required database/model work:

1. Add BusinessGroup model/table with ownership identity, optional legal/PAN metadata, INR currency, active state, and timestamps.
2. Extend Company with business_group_id, business_type (retail/cafe), stable slug, and is_demo.
3. Add company_id to Branch and change branch-name uniqueness to company scope.
4. Add business_group_id/company_id/token_version fields to User as specified by the TRD.
5. Extend UserRole with super_admin, order_taker, and kitchen while preserving existing role values.
6. Add or backfill company_id on every top-level confidential/operational record required by the TRD, including categories, suppliers, products, inventory, stock movements, customers, customer ledger/payments where selected, sales, invoices, purchase orders, forecasts, AI sessions, and audit logs.
7. Use parent-derived scope for child rows only where safe and documented.
8. Convert global uniqueness to company-aware uniqueness where appropriate:

- product SKU and barcode;
- branch name/code;
- invoice number;
- relevant customer/supplier identities.

9. Add company/branch/date/status indexes needed for scoped operational queries.
10. Add database/service checks preventing branch-company and cross-company foreign-key mismatches.

Migration strategy:

1. Use expand -> backfill -> validate -> contract Alembic migrations.
2. Add nullable columns before data backfill.
3. Create one default BusinessGroup and identify/create the Retail Company.
4. Backfill all current branches, users, and records to Retail.
5. Promote the existing seeded global Admin to super_admin with no company/branch.
6. Assign all normal users to Retail and preserve their branch assignments.
7. Validate row counts, sums, and relationship consistency.
8. Only after validation, apply non-null and final unique/check constraints.
9. Invalidate pre-migration auth tokens using token_version or the chosen documented strategy.
10. Do not rely on destructive downgrade for real data recovery; document backup-based rollback.

Seed work:

- Seed one business group.
- Keep all existing data in the Retail company.
- Seed one minimal Cafe company and branch with Non-GST intent, but no QR/order data.
- Do not duplicate Retail products or customers into Cafe.

Required tests to add or adapt:

- tests/test_multi_venture_schema.py
- tests/test_multi_venture_migrations.py
- tests/test_scope_constraints.py

Required cases:

- Clean migration to head succeeds.
- Upgrade from the previous seeded migration succeeds on PostgreSQL.
- Row counts and sales/invoice/inventory totals do not change during backfill.
- Every scoped record has a valid company after contract migration.
- Every branch-scoped row matches its branch company.
- Same SKU may exist in different companies but not twice in one company.
- Existing invoice and sale IDs/numbers remain stable.
- Invalid cross-company relationships are rejected.
- SQLite unit compatibility remains where practical, but PostgreSQL-specific behavior is tested on PostgreSQL.

Regression commands:

```
cd backend
alembic upgrade head
.\.venv\Scripts\python.exe -m pytest tests/test_multi_venture_schema.py tests/test_scope_constraints.py -q
```

```
.\.venv\Scripts\python.exe -m pytest tests/test_auth.py tests/test_master_data.py tests/test_inventory.py
tests/test_sales.py tests/test_invoices.py -q
.\.venv\Scripts\python.exe -m pytest -q
```

Manual reconciliation:

- Compare pre/post counts and totals from docs/MULTI_VENTURE_BASELINE.md.
- Inspect representative Retail company, branch, user, product, customer, invoice, sale, inventory, payment, and audit rows.
- Verify no orphan/mismatch validation query returns rows.

Do not:

- Add Cafe menu, tables, QR, orders, or customer portal.
- Add portal switching UI.
- Treat Admin as global in new schema design.
- Delete or rewrite existing financial records.
- Reset real data.

Acceptance criteria:

- Schema supports one business group and two companies.
- Existing data is reconciled to Retail with stable totals.
- Required constraints and indexes exist.
- Existing Retail tests pass.
- No Cafe functionality is exposed yet.

Report using the standard phase report and include:

- migration files and order;
- before/after row-count and financial-total reconciliation;
- schema decisions;
- PostgreSQL verification result;
- token invalidation behavior;
- any remaining nullable scope field and why.

Phase 2 Prompt: Venture Scope Enforcement, Roles, And Security Context

COPY-PASTE PROMPT

Copy and run this prompt only after Phase 1 migrations and reconciliation pass:

You are implementing Phase 2 of the approved multi-venture Retail and Cafe expansion.

Read before editing:

- PRD_MULTI_VENTURE_CAFE_EXPANSION.md, especially venture isolation and roles
- TRD_MULTI_VENTURE_CAFE_EXPANSION.md, especially Sections 4, 7, 10, and 14
- docs/MULTI_VENTURE_CAFE_IMPLEMENTATION_PHASES.md, Phase 2 and Stop Conditions
- docs/MULTI_VENTURE_BASELINE.md
- current auth dependencies, services, routes, schemas, exports, AI tools, tests, and migrations

Implement Phase 2 only: Venture Scope Enforcement, Roles, And Security Context.

Goal:

Make backend authorization a proven barrier between Retail and Cafe before any Cafe business feature is added.

Role contract:

- super_admin: only cross-venture role.
- admin: Venture Admin, company-wide within exactly one company.
- store_manager: assigned company/branch operations.
- staff: existing Retail operational role.
- order_taker: future Cafe order/billing role.
- kitchen: future preparation-only role.
- analyst: read-only within assigned company/branch.
- The Cafe partner will use admin with company_id set to Cafe.

Required backend work:

1. Introduce one immutable ScopeContext containing user, role, business group, company scope, branch IDs, and fixed permissions.
2. Build central dependencies/helpers for:

- current authenticated ScopeContext;
- required roles/permissions;
- company access;
- branch access;
- scoped object retrieval;
- Super Admin-only operations.

3. Reload active user and authoritative assignments from the database. Never trust role/company/branch from request payloads.
4. Update every existing protected module to use company plus branch scope:

- auth/me;

- business and GST settings;
- branches/categories/suppliers/products;
- customers and customer ledgers/payments;
- inventory and stock movements;
- sales and invoices/POS;
- purchase orders/reorder;
- dashboards and forecasts;
- exports and reporting;
- AI sessions and tool functions;
- audit access.

5. Query confidential objects by id plus allowed company/branch. Do not load by id and filter only in the frontend.
6. Validate every foreign object in a write belongs to the same company and allowed branch.
7. Make current Admin company-scoped. Only super_admin receives all-companies scope.
8. Make Analyst company/branch scoped rather than implicitly global.
9. Add token-version/session invalidation after role, company, branch, active-state, or password changes.
10. Add a short-lived step-up-authentication foundation for future GST activation and purge actions.
11. Use non-disclosing errors:

- cross-company object lookup normally returns generic not-found;
- forbidden route/capability returns structured 403;
- responses must not reveal another company name/count/id.

12. Audit repeated or high-risk denied cross-company access without logging secrets or unnecessary PII.

Required frontend compatibility work:

- Extend auth types with current company, role, branch scope, and permissions.
- Add generic forbidden/not-found handling.
- Do not build the separate portal shells yet.
- Continue to rely on backend responses as the authority.

Required test fixtures:

- One BusinessGroup.
- Retail and Cafe companies.
- Known records in each company for products, customers, inventory, sales, invoices, payments, purchase orders, forecasts, AI sessions, and audit.
- Super Admin, Retail Admin, Cafe Admin, branch users, and analysts.

Required tests:

- tests/test_company_scope_auth.py
- tests/test_cross_venture_isolation.py
- tests/test_cross_venture_exports_ai.py

Release-blocking negative cases:

1. Cafe Admin requests known Retail list/detail/create/update/deactivate records.
2. Retail user requests known Cafe records.

3. Same-company branch user requests another branch.
4. User manipulates company_id or branch_id in query/body.
5. Cafe Admin requests Retail dashboard, CSV export, AI answer/session, counts, search, or autocomplete.
6. Admin attempts Super Admin route/action.
7. Deactivated user/company uses an existing token.
8. User role/company changes while an old token is active.
9. Foreign IDs from another company are included in a valid write payload.
10. Pagination totals and error messages are checked for leakage.

Verification commands:

```
cd backend
.\.venv\Scripts\python.exe -m pytest tests/test_company_scope_auth.py
tests/test_cross_venture_isolation.py tests/test_cross_venture_exports_ai.py -q
.\.venv\Scripts\python.exe -m pytest -q
```

```
cd frontend
npm run typecheck
npm run build
```

Do not:

- Add Cafe menu/order features.
- Depend on frontend route hiding for security.
- Leave any protected existing module on branch-only or unscoped queries.
- accept user-supplied company scope as authority.

Acceptance criteria:

- Every protected module is company-aware.
- All known-object cross-venture tests fail closed.
- Admin and Analyst are not global.
- Only Super Admin receives cross-venture access.
- Full existing test suite and frontend checks pass.

This is a hard gate. If any cross-venture test succeeds, fix it and do not report the phase complete.

Report the modules audited, shared scope helpers introduced, negative test matrix, commands/results, and any residual security risk.

Phase 3 Prompt: Shared Login, Separate Portals, And Venture Users

COPY-PASTE PROMPT

Copy and run this prompt only after Phase 2 cross-venture tests pass:

You are implementing Phase 3 of the approved multi-venture Retail and Cafe expansion.

Read before editing:

- PRD_MULTI_VENTURE_CAFE_EXPANSION.md, Sections 6 through 9
- TRD_MULTI_VENTURE_CAFE_EXPANSION.md, Sections 4 and 13
- docs/MULTI_VENTURE_CAFE_IMPLEMENTATION_PHASES.md, Phase 3
- current auth API/context, App.tsx, navigation, layouts, pages, styles, seed data, and tests

Implement Phase 3 only: Shared Login, Separate Portals, And Venture User Management.

Goal:

Use one login URL while giving Final Super Admin, Retail users, and Cafe users completely different authorized portal experiences.

Required backend work:

1. Extend GET /auth/me to return only the current user's safe display scope:

- role;
- current company id/slug/name for normal users;
- allowed branch ids/names as needed;
- fixed permissions needed by UI;
- whether venture switching is allowed.

2. Add current-venture API and Super Admin-only venture list/summary APIs.
3. Add Super Admin user creation, assignment, activation/deactivation, and token invalidation behavior needed for venture users.
4. Enforce one company per normal user in the MVP.
5. Seed development users for:

- Final Super Admin;
- Retail Admin/Manager/Staff/Analyst;
- Cafe Partner Admin;
- Cafe Manager;
- Cafe Order Taker/Cashier;
- Cafe Kitchen Staff;
- Cafe Analyst.

6. Do not return the Retail company list/name to Cafe-scoped users.

Required frontend work:

1. Keep one shared /login page.
2. After authentication, fetch /auth/me and route using server-provided scope:

- super_admin -> /super-admin;
- Retail company user -> /retail;
- Cafe company user -> /cafe.

3. Add protected route shells:

- SuperAdminLayout for /super-admin/*;
- RetailLayout for /retail/*;
- CafeLayout for /cafe/*.

4. Add PublicOrderLayout route placeholder for /order/:qrToken, but do not implement ordering yet.
5. Show the active venture and branch clearly in authenticated top bars.
6. Show an All Ventures/Retail/Cafe selector only to Super Admin.
7. Cafe Partner Admin must have no venture selector and no Retail navigation.
8. Preserve or redirect current Retail routes without breaking the existing experience.
9. Add role-aware Cafe placeholders for:

- Dashboard;
- Live Orders;
- POS/New Order;
- Tables and QR;
- Menu;
- Billing;
- Payments/Closing;
- Inventory;
- Reports;
- Staff;
- Settings.

10. Kitchen navigation shows preparation queue placeholder only.
11. Use a restrained operational UI consistent with the current application. Do not build a landing/marketing page.

Required tests:

- tests/test_venture_users.py
- auth/scope regression tests from Phase 2
- frontend route/navigation tests if a test harness exists; otherwise document browser tests

Required browser checks:

1. Final Super Admin logs in and can choose All Ventures, Retail, or Cafe.
2. Cafe Partner Admin logs in and lands directly at /cafe/dashboard.
3. Cafe Partner sees no Retail name, menu item, route link, or venture selector.
4. Direct /retail URL and Retail API attempts are denied for Cafe Partner.
5. Retail Staff lands in /retail and sees no Cafe admin modules.
6. Kitchen user sees only the preparation surface.
7. Analyst remains read-only.
8. Unauthenticated users cannot access any authenticated shell.
9. Logout works from every shell and invalidates the token/session.

10. Desktop and tablet layouts do not overlap.

Verification commands:

```
cd backend
.\.venv\Scripts\python.exe -m pytest tests/test_auth.py tests/test_company_scope_auth.py
tests/test_venture_users.py -q
```

```
cd frontend
npm run typecheck
npm run build
```

Do not:

- Implement menu, QR, ordering, billing, or dashboards yet.
- Give Cafe Partner a client-side company switch.
- send all ventures to normal users and merely hide them.
- create separate authentication systems.

Acceptance criteria:

- One login routes every role to the correct portal.
- Cafe Partner experience is Cafe-only at UI and API levels.
- Super Admin can select and clearly see the active scope.
- Existing Retail app remains usable.

Report backend endpoints, seeded roles, frontend routes/layouts, browser checks, security results, and known gaps.

Phase 4 Prompt: GST-Ready, Non-GST-Default Configuration

COPY-PASTE PROMPT

Copy and run this prompt only after the separate portal and scope behavior is verified:

You are implementing Phase 4 of the approved multi-venture Retail and Cafe expansion.

Read before editing:

- PRD_HITECH_COMPETITIVE_ADDONS.md tax/invoice rules
- PRD_MULTI_VENTURE_CAFE_EXPANSION.md, requirements MV-FR-090 through MV-FR-099
- TRD_MULTI_VENTURE_CAFE_EXPANSION.md, Sections 5.11 and 11
- docs/MULTI_VENTURE_CAFE_IMPLEMENTATION_PHASES.md, Phase 4
- current Company, BusinessProfile, GSTRegistration, TaxRate, InvoiceSequence, Product, Customer, Invoice, POS, print-data, settings, seed, and test code

Implement Phase 4 only: GST-Ready, Non-GST-Default Configuration.

Goal:

Keep HSN/SAC and reference GST metadata internally while making current customer bills safely Non-GST until a valid registration and controlled activation exist.

Business contract:

- Both ventures may operate in Non-GST mode today.
- Non-GST bills must apply zero GST and display no GSTIN.
- Product HSN/SAC and reference GST rates may remain internal.
- Client requests cannot force GST mode.
- Future GST activation is effective-dated, Super Admin controlled, audited, and non-retroactive.
- The application monitors combined venture turnover for review but does not make a legal registration decision.

Required backend/database work:

1. Add/complete venture settings for:

- tax_registration_status: unregistered/registered;
- default_tax_mode: non_gst/gst;
- gst_effective_from;
- customer_details_on_bill: hidden/basic/full;
- b2b_gst_enabled;
- include_customer_in_gst_reports.

2. Validate settings at company/venture scope.
3. Set seeded Retail and Cafe operational defaults to Non-GST unless a specific test fixture activates GST.
4. Mark any demo GST registration as inactive/reference-only when the venture is unregistered. Never allow the existing demo GSTIN to leak onto a Non-GST bill.
5. Enforce server-side Non-GST invoice behavior:

- invoice type cannot be forced to GST;

- CGST/SGST/IGST/Cess totals are zero;
- no applied GST rows feed GST reporting;
- GSTIN is omitted from customer-facing invoice/print/PDF data;
- customer GST fields are omitted unless both venture and customer are explicitly enabled.

6. Keep product tax fields available only on permitted internal screens/APIs.
7. Add Super Admin-only GST activation workflow requiring:

- valid step-up authorization;
- registered status;
- active GST registration and GSTIN format validation assistance;
- legal/trade identity;
- state and state code;
- effective date;
- active GST invoice sequence/template;
- explicit confirmation acknowledging CA/GST review.

8. Block GST invoices dated before the activation effective date.
9. Never rewrite or convert historical Non-GST invoices when settings change.
10. Add owner-only combined-turnover monitoring grouped by the BusinessGroup. Label it as a review aid, not legal advice.
11. Audit old/new values, actor, scope, and time for every tax-mode change.

Required frontend work:

1. Show tax registration state and invoice mode in venture Settings.
2. Display Non-GST as the active default.
3. Disable GST billing controls while prerequisites are incomplete.
4. Hide GSTIN and customer GST sections from Non-GST receipt/preview data.
5. Keep HSN/reference GST visible only in authorized internal catalog/settings screens.
6. Add a clear compliance disclaimer and effective-date confirmation for future activation.
7. Cafe Partner may view the Cafe tax state but cannot activate GST unless future policy explicitly grants it; default is Super Admin only.

Required tests:

- tests/test_tax_operation_mode.py
- tests/test_non_gst_invoice_privacy.py
- tests/test_gst_activation_controls.py

Required cases:

1. Non-GST Retail and Cafe invoices have zero applied tax.
2. Non-GST customer outputs contain no GSTIN or customer GST data.
3. Internal product HSN/reference rate remains stored and role-scoped.
4. POS/client cannot force invoice_type=gst.
5. Venture Admin, Manager, Staff, Order Taker, and Analyst cannot activate GST.
6. Super Admin activation fails if any prerequisite is missing.
7. Successful test activation applies only on/after the effective date.
8. Existing Non-GST invoice remains byte/data-equivalent in tax fields after activation.

9. GST reporting excludes Non-GST invoice tax reference metadata.
10. Combined turnover includes Retail plus Cafe only for Super Admin.

Regression commands:

```
cd backend
.\.venv\Scripts\python.exe -m pytest tests/test_business_settings.py tests/test_invoice_tax.py
tests/test_invoices.py tests/test_pos_checkout.py -q
.\.venv\Scripts\python.exe -m pytest tests/test_tax_operation_mode.py
tests/test_non_gst_invoice_privacy.py tests/test_gst_activation_controls.py -q
.\.venv\Scripts\python.exe -m pytest -q
```

```
cd frontend
npm run typecheck
npm run build
```

Manual checks:

- Login as Super Admin and inspect each venture's tax state.
- Create a Non-GST Retail test bill and inspect API plus print/preview data.
- Login as Cafe Partner and confirm activation is unavailable.
- Attempt a forced GST checkout through API and confirm a structured denial.

Do not:

- Hardcode a legal registration threshold.
- Automatically activate GST based on turnover.
- Apply product reference tax to a Non-GST payable total.
- retroactively convert historical invoices.
- Build live GST portal/e-invoice/e-way integration.

Acceptance criteria:

- Accidental GST charging or GSTIN display is impossible in Non-GST mode.
- Product tax metadata remains internal and useful.
- Activation is guarded, effective-dated, audited, and Super Admin controlled.
- Existing invoice/POS behavior remains correct.

Report schema/settings changes, invoice behavior, activation controls, tests, compliance limitations, and seed changes.

Phase 5 Prompt: Cafe Menu, Tables, And Secure QR Foundation

COPY-PASTE PROMPT

Copy and run this prompt only after Phase 3 scope/portal controls pass. Phase 4 may run before or in parallel only if separate agents do not edit overlapping files; integrate and retest before completion.

You are implementing Phase 5 of the approved multi-venture Retail and Cafe expansion.

Read before editing:

- PRD_MULTI_VENTURE_CAFE_EXPANSION.md, requirements MV-FR-010 through MV-FR-035
- TRD_MULTI_VENTURE_CAFE_EXPANSION.md, Sections 5.6 through 5.8, 7.2, and 14.3
- docs/MULTI_VENTURE_CAFE_IMPLEMENTATION_PHASES.md, Phase 5
- existing products, inventory, company/branch scope, audit, file/print patterns, frontend layouts, seed data, and tests

Implement Phase 5 only: Cafe Menu, Tables, And Secure QR Foundation.

Goal:

Create company-scoped Cafe menu and table administration plus a secure, rotatable QR foundation. Do not accept customer orders yet.

Required models/migrations:

1. menu_categories:
- company_id;
- optional branch_id for company-wide or branch menu policy;
- name, display_order, active state, timestamps;
- company/branch-aware uniqueness.
2. menu_items:
- company_id, optional branch_id, category_id;
- optional same-company product_id;
- name, description, optional image reference;
- backend selling/customer display price;
- preparation_area: kitchen/beverage/counter/none;
- available and active states;
- display_order, version, timestamps.
3. cafe_tables:
- company_id, branch_id;
- unique branch table code and display name;
- optional capacity/area;
- active state, version, timestamps.
4. table_qr_tokens:
- explicit company/branch/table scope;
- opaque public reference where used;

- hashed secret token, non-secret prefix;
- expiry/revocation/last-used data;
- creator and timestamps.

5. table_sessions:

- opaque public id;
- company/branch/table scope;
- dine_in/takeaway/counter type;
- open/bill_requested/billed/closed/cancelled states;
- open/close actor and time;
- version;
- database protection against two active sessions for one table.

Required backend services/APIs:

- Cafe menu category list/create/update.
- Cafe menu item list/create/update/availability.
- Cafe table list/create/update/deactivate.
- QR rotate/revoke and print-data endpoint.
- Open/get/close table session for authorized staff.
- Scope all reads/writes to Cafe company and allowed branch.
- Validate linked products belong to the same company.
- Generate at least 256 bits of QR secret randomness.
- Store only a cryptographic hash; return raw secret only at creation/rotation for immediate QR rendering.
- Never serialize token_hash.
- Audit menu/table/QR/session administration.

Required frontend work:

1. Build Cafe Menu Management page with categories, search, availability, ordering, create/edit forms, and product link where appropriate.
2. Build Cafe Tables and QR page with table state, active session, rotate/revoke, preview, and printable QR data.
3. Add open/close session controls only for authorized roles.
4. Use existing operational design and responsive tablet/desktop layout.
5. Add loading, empty, error, conflict, and success states.
6. Do not show raw QR token after the one-time creation/rotation response is dismissed.

Seed requirements:

- Realistic Cafe categories such as Hot Beverages, Cold Beverages, Snacks, Breakfast, Meals, Desserts.
- 20 to 40 realistic menu items with Non-GST customer prices.
- 8 to 15 Cafe tables across sensible areas.
- Same-company product links for packaged/sellable items where useful.
- Development-only QR generation path clearly marked; no secrets in README or logs.

Required tests:

- tests/test_cafe_menu.py

- tests/test_cafe_tables.py
- tests/test_cafe_qr_security.py
- tests/test_table_sessions.py

Required cases:

1. Cafe Admin manages Cafe menu/tables only.
2. Retail and unauthorized roles cannot access Cafe administration.
3. Retail product cannot be linked to Cafe menu item.
4. Duplicate table code in one branch is rejected; same code in another branch may be allowed.
5. Negative prices and invalid preparation areas fail.
6. Token randomness/format is valid and raw token is not stored.
7. API never returns token_hash.
8. Revoked/expired token cannot resolve.
9. Concurrent active-session attempts create only one active table session.
10. Closing a session releases the table under valid rules.

Verification commands:

```
cd backend
alembic upgrade head
.\.venv\Scripts\python.exe -m pytest tests/test_cafe_menu.py tests/test_cafe_tables.py
tests/test_cafe_qr_security.py tests/test_table_sessions.py -q
.\.venv\Scripts\python.exe -m pytest tests/test_cross_venture_isolation.py tests/test_inventory.py
tests/test_products_retail_catalog.py -q
```

```
cd frontend
npm run typecheck
npm run build
```

Manual checks:

- Login as Cafe Partner Admin and manage a menu item/table.
- Rotate a QR and confirm only the new token resolves.
- Login as Retail Admin/Kitchen/Analyst and confirm prohibited writes fail.
- Open the table/QR pages at desktop and tablet sizes.

Do not:

- Implement public customer ordering.
- Create invoices or stock movements from table sessions.
- Store raw QR secrets.
- create a separate Cafe product catalog when a same-company product link is sufficient.

Acceptance criteria:

- Cafe menu and tables are fully company/branch scoped.
- QR lifecycle is secure and auditable.
- One active table session invariant is enforced by backend/database.
- No public ordering is enabled yet.

Report models, migration, APIs, UI, seed summary, token design, concurrency result, and known gaps.

Phase 6 Prompt: Customer Mobile QR Ordering

COPY-PASTE PROMPT

Copy and run this prompt only after Phase 5 QR and table-session tests pass:

You are implementing Phase 6 of the approved multi-venture Retail and Cafe expansion.

Read before editing:

- PRD_MULTI_VENTURE_CAFE_EXPANSION.md, customer journey and MV-FR-040 through MV-FR-047
- TRD_MULTI_VENTURE_CAFE_EXPANSION.md, Sections 5.8, 5.9, 7.3, 9.1, 10, 13.3, and 14.3
- docs/MULTI_VENTURE_CAFE_IMPLEMENTATION_PHASES.md, Phase 6 and Stop Conditions
- Phase 5 menu/table/QR/session code and tests

Implement Phase 6 only: Customer Mobile QR Ordering.

Goal:

Build a mobile customer portal opened from a Cafe table QR. The guest can view a safe menu, submit an idempotent order, view its current session status, add another order, and request the bill without logging in or accessing internal data.

Required models/migrations:

1. cafe_orders with opaque public id, company/branch/session scope, order number/type/source, controlled status, server totals, customer notes, nullable creator, idempotency hash, version, and timestamps.
2. cafe_order_items with menu/product links, name/SKU/price snapshots, quantity, discount/line total, item state, notes, source, billed link placeholder, version, and timestamps.
3. cafe_order_status_history with scope, old/new status, guest/actor marker, reason, and timestamp.
4. Add required unique constraints and indexes for company, session, status, date, and idempotency.

Required public security design:

1. QR resolve exchanges the static opaque QR secret for a short-lived guest session token.
2. Store only hashes of QR and guest secrets where persisted.
3. Guest token is tied to one table session, token version, and expiry.
4. Public URLs and responses use opaque UUID/random references, never sequential internal IDs.
5. Public requests cannot set company, branch, role, staff actor, price, discount, tax, status, or payment data.
6. Apply strict quantity/item-count/note-length/body-size limits.
7. Apply per-IP and per-token rate limiting suitable for local deployment/tunnel use.
8. Require Idempotency-Key on order submission.
9. Same key and same payload returns the original order; changed payload returns 409.
10. Render all customer notes as plain text and prevent HTML/script injection.

Required APIs:

- POST /public/cafe/qr/{opaque_token}/resolve
- GET /public/cafe/sessions/{public_id}/menu
- POST /public/cafe/sessions/{public_id}/orders
- GET /public/cafe/sessions/{public_id}/orders
- POST /public/cafe/sessions/{public_id}/bill-request

Public response rules:

- Return Cafe display name, table display identity, customer-safe menu, order public reference, safe status, and customer-facing totals.
- Do not return company ids, branch ids, internal table/session/order ids, stock quantities, costs, margins, tax configuration, user ids, token hashes, staff-only notes, customer lists, or other sessions.
- Use generic invalid/expired/revoked messages that do not disclose another record's existence.

Order behavior:

1. Backend validates active QR, company/branch/table/session, menu availability, and item quantities.
2. Backend recalculates all prices and totals.
3. Create order and item snapshots plus initial status history in one transaction.
4. Initial customer status is placed.
5. Staff acceptance is required in Phase 7 before preparation commitment.
6. Order placement does not create invoice, sale, payment, ledger entry, stock change, or stock movement.
7. Guest may submit additional orders to the same open session using new idempotency keys.
8. Guest may request bill but cannot mark it paid or closed.

Required frontend work:

1. Build /order/:qrToken as the actual mobile ordering experience, not a landing page.
2. Add QR resolving, table confirmation, category navigation, search, menu items, availability, cart, quantities, notes, total, submit, and order/session status.
3. Keep the Idempotency-Key stable through retry until a final response.
4. Poll safe order/session status every 5 to 10 seconds while visible; slow/pause when hidden.
5. Support adding another order and requesting the bill.
6. Add clear invalid QR, revoked QR, expired guest session, closed table, unavailable item, duplicate retry, rejected order, backend unavailable, and session closed states.
7. Ensure common Android/iOS widths have no overlap or horizontal scrolling.
8. Do not expose login/admin navigation to customers.

Required tests:

- tests/test_public_qr_orders.py
- tests/test_public_qr_authorization.py
- tests/test_order_idempotency.py
- tests/test_public_rate_limits.py

Release-blocking cases:

1. Guest changes session/order public id and cannot see another table.
2. Guest adds company_id/branch_id/internal id and gains nothing.
3. Guest modifies price, discount, tax, or status and backend ignores/rejects it.
4. Duplicate request creates exactly one order.
5. Same key with different payload returns conflict.
6. Revoked/expired QR or guest token fails.
7. Closed session cannot accept items.
8. Limits and rate limiting work.

9. Public JSON contains no confidential/internal fields.
10. Script/HTML note input is safely handled.
11. Customer order creates no stock movement or revenue record.

Verification commands:

```
cd backend
alembic upgrade head
.\.venv\Scripts\python.exe -m pytest tests/test_public_qr_orders.py tests/test_public_qr_authorization.py
tests/test_order_idempotency.py tests/test_public_rate_limits.py -q
.\.venv\Scripts\python.exe -m pytest tests/test_cafe_qr_security.py tests/test_table_sessions.py
tests/test_cross_venture_isolation.py -q
```

```
cd frontend
npm run typecheck
npm run build
```

Manual browser verification:

- Use phone-sized and desktop viewports.
- Scan/open a valid QR, add multiple items, submit, retry during a simulated slow network, and confirm one order.
- Try revoked QR and closed session.
- Request bill and confirm payment cannot be changed by the guest.
- Inspect network responses for confidential fields.

Do not:

- Add staff/kitchen status controls.
- reduce inventory.
- issue an invoice.
- add customer online payment.
- expose another table's shared data without the valid guest session.

Acceptance criteria:

- Customer can place a real database-backed Cafe order without login.
- Public attack cases fail closed.
- Retry is idempotent.
- No stock, invoice, payment, or revenue effect occurs yet.

Report public API design, token/session behavior, limits, idempotency, mobile UI, tests, and residual abuse risks.

Phase 7 Prompt: Staff Order Entry, Unified Queue, And Kitchen Workflow

COPY-PASTE PROMPT

Copy and run this prompt only after Phase 6 public security tests pass:

You are implementing Phase 7 of the approved multi-venture Retail and Cafe expansion.

Read before editing:

- PRD_MULTI_VENTURE_CAFE_EXPANSION.md, MV-FR-050 through MV-FR-065
- TRD_MULTI_VENTURE_CAFE_EXPANSION.md, Sections 7.4, 8, 10, and 13.2 through 13.4
- docs/MULTI_VENTURE_CAFE_IMPLEMENTATION_PHASES.md, Phase 7
- existing ScopeContext, Cafe menu/table/session/public order services, role permissions, UI shells, and tests

Implement Phase 7 only: Staff Order Entry, Unified Queue, And Kitchen Workflow.

Goal:

Allow authenticated Cafe staff to enter dine-in/takeaway/counter orders and process customer QR plus staff orders through one secure live queue and controlled state machine.

Required backend work:

1. Add authenticated order list/create/detail APIs scoped to Cafe company and allowed branch.
2. Staff order creation must reuse the same order/item pricing and snapshot service used by customer QR ordering.
3. Support order types:

- dine_in with an active table session;
- takeaway;
- counter.

4. Preserve source_channel as qr_customer, order_taker, billing_counter, or manager.
5. Preserve creator/actor where authenticated and guest marker where public.
6. Implement explicit service methods/endpoints for:

- accept;
- reject with reason;
- start preparing;
- mark ready;
- serve;
- request bill;
- cancel with role and reason rules.

7. Do not expose a generic arbitrary status-update endpoint.
8. Validate the order state machine from the TRD and reject invalid transitions.
9. Add optimistic concurrency/version checks and 409 stale_state responses.
10. Add live queue filters for branch, table, status, source, preparation area, business date, and unbilled state, intersected with server scope.

11. Kitchen role receives only preparation-relevant items, table/order reference, quantities, notes, age, and allowed preparation actions.
12. Kitchen must not receive customer ledger, payment, cost, margin, tax setting, Retail, or owner-report data.
13. Add audit/status history for every transition and cancellation.
14. Keep order placement and preparation free of invoice/stock/revenue effects in this MVP.

Required frontend work:

1. Build Cafe Live Orders page for Partner Admin, Manager, and Order Taker.
2. Build staff New Order/POS order-entry page using menu search/category/cart and backend prices.
3. Add table selector and takeaway/counter modes.
4. Show source channel, table, age, status, preparation area, and notes clearly.
5. Build Kitchen queue with only kitchen-relevant controls and information.
6. Add explicit action buttons for valid next states; do not use a free-form status dropdown.
7. Add rejection/cancellation reason dialogs.
8. Poll active queue about every five seconds while visible, cancel stale requests, and refetch after actions.
9. Handle stale state/conflict without overwriting another user's update.
10. Preserve dense operational layout on desktop/tablet with no card nesting or oversized marketing elements.

Required tests:

- tests/test_cafe_staff_orders.py
- tests/test_cafe_order_transitions.py
- tests/test_cafe_order_permissions.py
- tests/test_cafe_order_concurrency.py

Required cases:

1. QR and staff orders appear in one queue.
2. Staff order uses the same backend price logic as QR order.
3. Cafe roles cannot cross company/branch.
4. Kitchen sees only preparation data and allowed actions.
5. Analyst is read-only; Retail roles cannot use Cafe order endpoints.
6. Every invalid state transition is rejected.
7. Two actors updating the same version result in one success and one conflict.
8. Rejection/cancellation requires reason and creates history/audit.
9. Additional staff order joins the same table session without rewriting earlier orders.
10. No invoice, payment, sale, stock movement, or inventory reduction occurs.

Verification commands:

```
cd backend
.\.venv\Scripts\python.exe -m pytest tests/test_cafe_staff_orders.py tests/test_cafe_order_transitions.py tests/test_cafe_order_permissions.py tests/test_cafe_order_concurrency.py -q
.\.venv\Scripts\python.exe -m pytest tests/test_public_qr_orders.py tests/test_cross_venture_isolation.py -q
```

```
cd frontend
npm run typecheck
npm run build
```

Manual browser flow:

- Submit one QR order.
- Login as Order Taker, create another order for the same table, and accept both.
- Login as Kitchen and move preparation items through preparing/ready.
- Login as Manager/Order Taker and mark served.
- Trigger a stale update from two browser sessions and verify conflict handling.
- Confirm customer status polling reflects safe state changes.

Do not:

- Generate bills or payments yet.
- reduce stock during ordering/preparation.
- add WebSockets/Redis.
- expose financial/customer data to Kitchen.

Acceptance criteria:

- QR and staff orders share one queue and service layer.
- State transitions, permissions, history, and concurrency are enforced by backend.
- Kitchen has a minimal safe operational view.
- Existing Retail behavior remains unchanged.

Report APIs, state machine, permissions, concurrency strategy, UI pages, polling behavior, tests, and known gaps.

Phase 8 Prompt: Cafe Billing, Payments, And Table Closing

COPY-PASTE PROMPT

Copy and run this prompt only after Phases 4 and 7 pass:

You are implementing Phase 8 of the approved multi-venture Retail and Cafe expansion.

Read before editing:

- PRD_HITECH_COMPETITIVE_ADDONS.md invoice, payment, stock, ledger, and print rules
- PRD_MULTI_VENTURE_CAFE_EXPANSION.md, MV-FR-070 through MV-FR-085
- TRD_MULTI_VENTURE_CAFE_EXPANSION.md, Sections 5.10, 9.3, 10, and 11
- docs/MULTI_VENTURE_CAFE_IMPLEMENTATION_PHASES.md, Phase 8
- current Invoice/POS, Sales compatibility, PaymentMode, Customer Ledger, Inventory, StockMovement, Audit, Cafe order/session, tax-mode, and frontend POS code

Implement Phase 8 only: Cafe Billing, Payments, And Table Closing.

Goal:

Convert eligible unbilled Cafe order items into exactly one correct invoice, payment result, stock effect, and table close using the existing shared financial engines.

Critical rules:

- Backend calculates every total.
- One table session may contain multiple QR and staff orders.
- Each eligible order item is billed exactly once.
- Order placement did not reduce stock; invoice issue applies configured stock effect once.
- Every stock change creates a stock movement.
- Invoice, sale compatibility, stock, payments, ledger, Cafe billed links, status, and audit commit or roll back together.
- Current Non-GST mode must produce zero applied GST and no GSTIN customer output.

Required database/model work:

1. Ensure Invoice has non-null company_id and Cafe source fields:
- source_type: cafe_table_session/cafe_takeaway as applicable;
- source_id;
- idempotency hash.
2. Add safe uniqueness preventing more than one active invoice for one Cafe billing source.
3. Add billed_invoice_id/billed_invoice_item_id links on Cafe session/order/items as designed.
4. Add indexes for company, branch, source, invoice date/status, and unbilled item lookup.
5. Preserve existing invoice/sale IDs and Retail compatibility.

Required backend service work:

1. Add table-session bill review/quote that returns backend-calculated eligible items and totals without mutating data.
2. Add idempotent bill endpoint for table sessions.

3. Add direct takeaway/counter bill endpoint or integrate it cleanly with the same service.
4. In one transaction:

- scope and lock/version-check the session;
- select only accepted/served eligible unbilled items;
- reject invalid/cancelled/already billed items;
- validate idempotency and existing source invoice;
- recalculate prices, discounts, tax mode, round-off, and total;
- generate the company/branch invoice number safely;
- create invoice/items/status/tax rows according to active mode;
- create/reuse analytics-compatible Sale linkage without double counting;
- lock and update linked product inventory once;
- create stock movements for every inventory change;
- create initial invoice payments and any allowed customer ledger effect;
- link Cafe items to invoice items;
- mark relevant orders/session billed;
- write audit evidence;
- commit all effects together.

5. Support existing Cash, UPI, Card, Bank, Credit, and valid split-payment rules.
6. Validate payment references where the payment mode requires one.
7. Prevent paid closure when recorded payments/credit do not satisfy the invoice policy.
8. Close/release the table only after valid billing/payment state.
9. A retry with the same idempotency key returns the original invoice. A changed payload returns conflict.
10. If invoice print/PDF remains unfinished, implement the minimum shared browser-printable Non-GST receipt/print-data prerequisite needed for both Retail and Cafe. Do not create a second Cafe-only total or template engine. Keep full advanced template work separate if not required for this flow.

Required frontend work:

1. Add table bill review showing each source order and unbilled item.
2. Make already billed/cancelled items unmistakable and non-selectable.
3. Add backend quote refresh before confirmation.
4. Add payment mode and supported split-payment controls.
5. Prevent duplicate checkout clicks and keep the idempotency key stable through retry.
6. Show invoice success, number, total, payments, balance, and table-close result.
7. Add print/download action using the shared invoice print implementation available after this phase.
8. Add direct takeaway/counter billing flow.
9. Handle insufficient stock, changed menu/order, stale session, duplicate billing, payment mismatch, and backend rollback errors clearly.

Required tests:

- tests/test_cafe_billing.py
- tests/test_cafe_billing_idempotency.py
- tests/test_cafe_billing_stock.py
- tests/test_cafe_payments.py

- tests/test_cafe_non_gst_bills.py

Release-blocking cases:

1. One QR order plus one staff order creates one bill containing every eligible item exactly once.
2. Two simultaneous/retried bill requests create one active invoice.
3. Same idempotency key returns original invoice; changed payload conflicts.
4. Linked inventory decreases once and matching stock movements exist.
5. Unlinked/prepared-food item follows documented no-stock behavior without fake ingredient tracking.
6. Failure after invoice creation but before payment/stock completion rolls back everything.
7. Payment sum, invoice paid amount, balance, and ledger reconcile.
8. Non-GST bill has zero applied tax and no GSTIN/customer GST output.
9. Existing Retail invoice/POS/dashboard does not double-count invoice-linked sales.
10. Already billed/cancelled item cannot be billed again.
11. Table closes and can open a new session only after valid settlement.
12. Unauthorized roles/branches/ventures cannot quote, bill, pay, or close.

Verification commands:

```
cd backend
alembic upgrade head
.\.venv\Scripts\python.exe -m pytest tests/test_cafe_billing.py tests/test_cafe_billing_idempotency.py
tests/test_cafe_billing_stock.py tests/test_cafe_payments.py tests/test_cafe_non_gst_bills.py -q
.\.venv\Scripts\python.exe -m pytest tests/test_invoices.py tests/test_pos_checkout.py
tests/test_invoice_tax.py tests/test_inventory.py tests/test_sales.py tests/test_dashboard.py
tests/test_customer_ledger.py -q
.\.venv\Scripts\python.exe -m pytest -q
```

```
cd frontend
npm run typecheck
npm run build
```

Manual end-to-end flow:

- Open Cafe table session.
- Submit customer QR order.
- Add staff order to same table.
- Accept/prepare/serve.
- Review backend quote.
- Pay with Cash/UPI or a valid split.
- Confirm one invoice, correct receipt, one stock effect, correct dashboard source data, and released table.
- Repeat the checkout request and confirm no duplicate effects.
- Run a takeaway sale.

Do not:

- Recalculate authoritative totals only in React.
- create a separate Cafe invoice table/engine.
- decrement stock both at order and invoice stages.
- implement recipe ingredient consumption.
- close a table with unhandled payment balance.

Acceptance criteria:

- Complete Cafe order-to-bill-to-payment-to-close workflow works.
- Financial/stock operations are atomic and idempotent.
- Non-GST rules are enforced.
- Existing Retail POS and analytics remain correct.

Report transaction boundaries, invoice/source links, stock behavior, payment behavior, print status, tests, reconciliation evidence, and known gaps.

Phase 9 Prompt: Cafe And Consolidated Dashboards, Exports, And AI

COPY-PASTE PROMPT

Copy and run this prompt only after Phase 8 billing reconciliation passes:

You are implementing Phase 9 of the approved multi-venture Retail and Cafe expansion.

Read before editing:

- PRD.md dashboard, export, Power BI, forecasting, and AI rules
- PRD_MULTI_VENTURE_CAFE_EXPANSION.md, MV-FR-100 through MV-FR-115 and reporting definitions
- TRD_MULTI_VENTURE_CAFE_EXPANSION.md, Sections 15, 16, and 18
- docs/MULTI_VENTURE_CAFE_IMPLEMENTATION_PHASES.md, Phase 9
- current dashboard, exports, reporting views, AI tool router, invoice/sales compatibility, payment, Cafe order, and frontend chart code

Implement Phase 9 only: Cafe And Consolidated Dashboards, Exports, And AI.

Goal:

Give the Cafe Partner exact Cafe-only operational/financial visibility and give Final Super Admin separate Retail, Cafe, and consolidated views without data leakage or double counting.

Metric definitions that must remain separate:

- ordered value: submitted Cafe order value, including unbilled where selected;
- billed revenue: issued invoice value before approved returns/voids according to report definition;
- net billed revenue: billed revenue less approved return/refund/credit effects;
- collections: recorded non-credit payments adjusted for refunds;
- outstanding: invoice/customer-ledger amount not collected;
- cancelled value: order value cancelled before billing;
- void value: value reversed after issue;
- expected cash and variance: based on recorded cash movements, not total invoice value.

Required backend work:

1. Create a Cafe dashboard service layer with date, branch, source, status, payment mode, menu category/item, and table filters where relevant.
2. Add Cafe KPIs:

- order count and ordered value;
- billed/net revenue;
- collections and outstanding;
- average bill value;
- cancellations/refunds/voids;
- top menu items;
- source-channel mix;
- payment-mode mix;

- table turnover/session duration;
- open/unbilled sessions;
- low-stock linked products where useful.

3. Add Super Admin consolidated dashboard with explicit All Ventures/Retail/Cafe scope.
4. Normalize Retail and Cafe revenue sources so invoice-linked sales are counted once.
5. Add venture-aware reporting views/services:

- vw_cafe_order_summary;
- vw_cafe_order_items;
- vw_cafe_table_turnover;
- vw_cafe_billing_reconciliation;
- vw_cafe_payment_summary;
- vw_venture_sales_summary;
- vw_business_group_turnover.

6. Add scoped CSV exports with clear venture/branch/date/status headers.
7. Cafe Partner exports contain Cafe rows only and should not need a hidden Retail filter.
8. Super Admin consolidated exports include explicit venture identity.
9. Extend AI with safe database-backed tools:

- get_cafe_sales_summary;
- get_open_table_sessions;
- get_pending_cafe_orders;
- get_cafe_payment_reconciliation;
- get_cafe_top_items;
- get_cafe_cancelled_items;
- get_venture_comparison for Super Admin only.

10. AI tools receive ScopeContext from backend. Ignore any company requested by the model/user unless Super Admin's validated filter allows it.
11. Store scope labels in tool output so the formatter cannot mix ventures.
12. Continue deterministic fallback when no OpenAI key exists.

Required frontend work:

1. Build Cafe dashboard with concise KPI row, trends, order funnel/status, payment mix, top items, open tables, cancellations, and reconciliation tables.
2. Build Super Admin consolidated dashboard with unmistakable active scope selector and venture comparison.
3. Every dashboard/export page shows active venture, branch, and date range.
4. Add Cafe report/export controls.
5. Add Cafe AI suggested questions, such as:

- What are today's Cafe billed sales?
- How much money was collected today?
- Which tables are still open?
- Which Cafe items sold most this week?
- Show cancelled Cafe items today.

6. Do not show the venture comparison question to Cafe-scoped users.

7. Add loading, empty, error, stale-filter, and no-data states.

Required tests:

- tests/test_cafe_dashboard.py
- tests/test_consolidated_dashboard.py
- tests/test_multi_venture_exports.py
- tests/test_cafe_ai_scope.py

Required reconciliation/security cases:

1. Cafe Partner dashboard/export/AI includes Cafe only.
2. Known Retail data does not alter Cafe counts, search, pagination, or AI text.
3. Super Admin Cafe filter equals Cafe source totals.
4. Consolidated net billed revenue equals Retail plus Cafe for the identical period/filter.
5. Cancelled unbilled orders do not count as revenue.
6. Billed but unpaid value affects revenue/outstanding, not collections.
7. Collections equal payment rows adjusted for refunds.
8. Invoice-linked Sale is not counted twice.
9. Export row count/totals reconcile to API/dashboard.
10. AI numerical answers cite tool data and cannot select another venture for Cafe Partner.
11. No-key deterministic AI fallback remains accurate.

Verification commands:

```
cd backend
.\.venv\Scripts\python.exe -m pytest tests/test_cafe_dashboard.py tests/test_consolidated_dashboard.py tests/test_multi_venture_exports.py tests/test_cafe_ai_scope.py -q
.\.venv\Scripts\python.exe -m pytest tests/test_dashboard.py tests/test_exports.py tests/test_ai.py tests/test_sales.py tests/test_invoices.py -q
.\.venv\Scripts\python.exe -m pytest -q
```

```
cd frontend
npm run typecheck
npm run build
```

Manual verification:

- Record a known Retail sale and Cafe bill/payment.
- Compare source rows, Cafe dashboard, Super Admin Cafe filter, consolidated dashboard, CSV, and AI.
- Login as Cafe Partner and deliberately ask for Retail comparison; confirm no Retail disclosure.
- Test empty date range and refunded/cancelled examples.

Do not:

- Hardcode KPI values.
- treat orders, invoices, and collections as the same metric.
- trust frontend company filters.
- allow AI to invent or infer hidden venture numbers.
- introduce a paid BI dependency.

Acceptance criteria:

- Cafe and consolidated metrics reconcile exactly to source data.

- Cafe Partner sees Cafe only in UI, API, exports, and AI.
- Super Admin can compare ventures without weakening normal-user scope.
- Existing Retail dashboards/exports/AI still pass.

Report KPI formulas, query/view changes, API/UI, export schema, AI tools, reconciliation evidence, and residual limitations.

Phase 10 Prompt: Daily Closing, Audit, Void, And Controlled Purge

COPY-PASTE PROMPT

Copy and run this prompt only after Phase 9 reports reconcile:

You are implementing Phase 10 of the approved multi-venture Retail and Cafe expansion.

Read before editing:

- PRD_MULTI_VENTURE_CAFE_EXPANSION.md, MV-FR-120 through MV-FR-128
- TRD_MULTI_VENTURE_CAFE_EXPANSION.md, Sections 5.12, 8.3, 9.4, 9.5, and 12
- docs/MULTI_VENTURE_CAFE_IMPLEMENTATION_PHASES.md, Phase 10
- current invoice cancel, payment, customer ledger, inventory/stock movement, audit, backup/restore, authentication, reporting, and Cafe billing code

Implement Phase 10 only: Daily Closing, Audit, Void, And Controlled Purge.

Goal:

Provide accurate day-end accountability and safe correction/removal workflows. Final Super Admin receives exceptional purge authority, but no user receives a silent-delete path that breaks stock, payment, ledger, tax, or audit history.

Required database/model work:

1. business_day_closures with company/branch/date uniqueness, opening/expected/counted cash, component amounts, variance, status, actors, reasons, and timestamps.
2. record_purge_requests with scope, entity allowlist type/id, reason, status, actors, backup reference, dependency report, approvals, and timestamps.
3. record_tombstones with immutable non-sensitive evidence, before hash, scope, reason, actors, and purge time.
4. Add links between original financial records and their reversal/refund/credit/stock compensation records where missing.
5. Add indexes for company/branch/date/status.

Required daily-closing work:

1. Calculate expected cash from opening cash plus cash collections minus cash refunds and approved cash expenses, using available modules only.
2. Keep non-cash payment modes separate.
3. Support open -> submitted -> closed and controlled reopened state.
4. Require counted cash and show variance.
5. Limit submit/close/reopen actions by role and branch/company scope.
6. Reopen requires step-up/authorized role, reason, audit, and affected-report refresh.
7. Block direct mutations in closed periods according to policy; corrections use reversal workflows.

Required void/reversal work:

1. Implement allowlisted services for supported issued invoices/payments/stock/ledger graphs.
2. Validate actor, scope, state, date/period, dependencies, and reason.
3. Never delete original stock movements, payments, or ledger entries during normal correction.
4. Create compensating stock movement for every stock reversal.

5. Create payment refund/reversal and ledger compensation as required.
6. Preserve original record and link its reversal.
7. Update dashboards/reports through statuses and compensation, not hidden mutation.
8. Make the transaction atomic and idempotent.

Required controlled-purge work:

1. Only Final Super Admin may create/approve/execute a purge request.
2. Require a short-lived step-up grant scoped to the purge action.
3. Require a non-empty reason and explicit entity/scope from an allowlist.
4. Generate a dependency report before approval.
5. Require a verified backup reference and current restore-check policy.
6. Check business/fiscal period and configured retention lock.
7. Require optional second approval from a different eligible person when configured.
8. Require typed confirmation tied to the request.
9. Execute through entity-specific handlers. Do not provide raw SQL, arbitrary table name, generic recursive delete, or database shell access in the UI/API.
10. Purge the complete approved dependency graph or fail safely; do not orphan balances.
11. Write immutable tombstone and final audit evidence outside the purged graph.
12. Audit/tombstone records have no application delete endpoint.
13. Demo reset is a separate operation enabled only when environment and company is_demo both allow it.

Required frontend work:

1. Build Cafe daily closing/reconciliation page.
2. Show payment-mode totals, expected cash, counted cash, and variance clearly.
3. Build reason-based void/refund/correction actions for authorized users.
4. Build Super Admin purge request list/detail with dependency report, backup reference, approval, step-up, and typed confirmation.
5. Display irreversible consequences clearly without exposing internal table names/SQL.
6. Cafe Partner can see permitted Cafe audit/void history but cannot permanently purge.

Required tests:

- tests/test_business_day_closing.py
- tests/test_financial_voids.py
- tests/test_purge_workflow.py
- tests/test_purge_authorization.py
- tests/test_audit_immutability.py

Release-blocking cases:

1. Cafe Partner/Admin/Manager cannot execute permanent purge.
2. Super Admin cannot skip step-up, reason, dependency, backup, period, approval, or typed-confirmation gates.
3. Second approval cannot be supplied by the requester when enabled.
4. Void restores/reverses stock exactly once with movements.
5. Payment refund/reversal and ledger balance reconcile.
6. Locked/closed day rejects prohibited direct mutation.

7. Reopen requires authorized reason and audit.
8. Purge handler refuses unknown entity types and unsafe graphs.
9. Completed disposable-data purge leaves tombstone/audit evidence.
10. Audit/tombstone delete API does not exist.
11. Partial execution failure rolls back or produces a safe failed state with recovery instructions.
12. Demo reset is unavailable in production/non-demo configuration.

Verification commands:

```
cd backend
alembic upgrade head
.\.venv\Scripts\python.exe -m pytest tests/test_business_day_closing.py tests/test_financial_voids.py
tests/test_purge_workflow.py tests/test_purge_authorization.py tests/test_audit_immutability.py -q
.\.venv\Scripts\python.exe -m pytest tests/test_cafe_billing.py tests/test_inventory.py
tests/test_customer_ledger.py tests/test_dashboard.py -q
.\.venv\Scripts\python.exe -m pytest -q
```

```
cd frontend
npm run typecheck
npm run build
```

Operational verification must use disposable data:

- Create a backup.
- Restore to a separate disposable PostgreSQL database.
- Run one allowed void and verify reconciliation.
- Run one approved purge workflow and verify tombstone/audit/integrity.
- Never run purge testing against the working business database.

Do not:

- Add a generic DELETE endpoint for issued financial records.
- let Super Admin bypass integrity, retention, backup, or audit gates.
- delete audit logs/tombstones.
- hardcode secrets in backup commands.

Acceptance criteria:

- Daily cash and non-cash reconciliation is accurate.
- Normal corrections use compensating records.
- Exceptional purge exists but is tightly controlled and auditable.
- Backup/restore and post-action integrity are demonstrated.

Report models/migration, formulas, void handlers, purge state machine, security controls, disposable restore/purge evidence, tests, and legal/retention limitations.

Phase 11 Prompt: Security Hardening, End-To-End QA, And Release Packaging

COPY-PASTE PROMPT

Copy and run this final prompt only after Phases 0 through 10 are complete:

You are implementing Phase 11, the final verification and release-hardening phase for the approved multi-venture Retail and Cafe expansion.

Read every source of truth and the completed phase reports:

- PRD.md
- EXECUTION_FLOW_ANALYSIS.md
- PRD_HITECH_COMPETITIVE_ADDONS.md
- PRD_MULTI_VENTURE_CAFE_EXPANSION.md
- TRD_MULTI_VENTURE_CAFE_EXPANSION.md
- docs/MULTI_VENTURE_CAFE_IMPLEMENTATION_PHASES.md
- AGENT_STEP_BY_STEP_PROMPTS_MULTI_VENTURE_CAFE.md
- docs/MULTI_VENTURE_BASELINE.md
- README.md and all setup/security/backup/remote-access/QA docs
- current code, migrations, seed, tests, dependency files, and git status

Implement Phase 11 only: Security Hardening, End-To-End QA, And Release Packaging.

Goal:

Prove the complete two-venture product works securely from login or QR scan through database effects, reporting, audit, backup, and recovery. Do not add unrelated features.

Security hardening:

1. Finalize MFA/TOTP for Super Admin and Venture Admin before public remote production access, with development-only bypass disabled by default outside development.
2. Confirm short-lived access tokens, token-version revocation, logout, role/company changes, and step-up grants.
3. Confirm exact production CORS origins; remove broad wildcard behavior.
4. Add/verify security headers through application or documented reverse proxy:

- Content Security Policy appropriate to the React app;
- frame restrictions;
- content-type protections;
- referrer policy;
- HTTPS behavior.

5. Verify rate limits for login, step-up, QR resolve, public order submit, and bill request.
6. Disable or protect production OpenAPI/docs endpoints.
7. Review logs/errors for passwords, tokens, QR secrets, database credentials, payment secrets, stack traces, and unnecessary PII.
8. Verify PostgreSQL binds only to local/private interfaces and is not exposed by the tunnel.
9. Verify least-privilege application database credentials and separate migration privilege where practical.

10. Review dependencies for known vulnerabilities using available ecosystem tools; make only safe, tested updates required for release.
11. Confirm backup files are outside web/static roots and protected.

Automated QA:

1. Run full backend suite in the normal fast test configuration.
2. Run PostgreSQL integration tests for migrations, partial indexes, concurrency/locking, invoice sequence, idempotency, and scoped constraints.
3. Test a clean Alembic install to head.
4. Test upgrade from the pre-expansion baseline to head using disposable seeded data.
5. Run frontend typecheck and production build.
6. Add/run browser end-to-end automation for critical journeys if not already present.
7. Test desktop, tablet, and mobile viewports.
8. Test slow network, duplicate click, request timeout/retry, backend outage, expired session, revoked QR, and stale order state.
9. Verify CSV exports and AI fallback/provider behavior.
10. Perform backup and restore drill into a separate database.

Required end-to-end scenarios:

1. Final Super Admin logs in with hardened authentication and sees All Ventures, Retail, and Cafe scopes.
2. Cafe Partner Admin logs in and sees Cafe only.
3. Cafe Partner attempts known Retail URLs, object IDs, searches, dashboards, exports, AI requests, and receives no Retail data or identifying metadata.
4. Retail user attempts known Cafe records and is denied.
5. Customer scans valid table QR on mobile, views safe menu, and submits an order.
6. Customer retry creates one order.
7. Order Taker accepts the QR order and adds a staff order to the same table.
8. Kitchen sees only preparation data and updates the order.
9. Cashier bills all eligible items once, records payment, prints receipt, and closes the table.
10. Inventory and stock movements reconcile exactly once.
11. Cafe dashboard, Super Admin Cafe filter, CSV export, and AI return the same billed/collected values.
12. Consolidated dashboard equals Retail plus Cafe without double count.
13. Non-GST receipt contains zero applied GST, no GSTIN, and no hidden customer GST leakage.
14. Daily close reconciles expected/count cash and variance.
15. Authorized void creates compensating records and adjusted reports.
16. Unauthorized purge fails.
17. Controlled purge succeeds only on disposable data after every gate and leaves tombstone/audit evidence.
18. Backup restores and the restored app passes health/integrity checks.

Documentation and packaging:

1. Update README project name and clearly describe current implemented features.
2. Update architecture diagrams with BusinessGroup -> Company -> Branch and all four portal routes.
3. Update setup guide, migrations, seed commands, and all demo credentials.
4. Add Cafe table/QR setup and operations guide.

5. Update remote-access documentation to distinguish public customer order routes from authenticated admin routes and to keep PostgreSQL private.
6. Update backup/restore guide with multi-venture verification and purge prerequisites.
7. Update QA checklist with every role and cross-venture scenario.
8. Update demo script for Final Super Admin, Cafe Partner, customer QR, staff, kitchen, billing, dashboard, and reconciliation.
9. Add docs/MULTI_VENTURE_FINAL_VERIFICATION.md mapping every MV-FR requirement to:

- implementation evidence;
- automated test;
- manual/browser evidence;
- pass/fail status;
- remaining non-blocking gap.

10. Separate production-ready functions from portfolio/demo GST or compliance aids.

Final commands:

```
cd backend
alembic upgrade head
.\.venv\Scripts\python.exe -m pytest -q
```

```
cd frontend
npm run typecheck
npm run build
```

Also run the configured PostgreSQL integration and browser E2E commands and list them exactly in the report.

Do not:

- Add loyalty, delivery, recipe inventory, WebSockets, Redis, online gateway, native app, SaaS tenancy, or other deferred scope.
- hide failed tests.
- claim production readiness when a critical/high authorization issue remains.
- expose the database or secrets for remote access.

Final acceptance criteria:

- Every release-blocking test passes.
- No critical/high cross-venture authorization issue remains open.
- Cafe Partner is technically and visually Cafe-only.
- Customer QR and staff orders converge into one correct bill.
- Retail regression passes.
- Non-GST and future activation controls work.
- Dashboards/exports/AI reconcile to source data.
- Void/purge/closing controls preserve audit and integrity.
- Backup/restore succeeds.
- Final verification document maps all MV-FR requirements.

Report using the standard phase format plus:

- complete command list and results;
- test counts by layer;

- migration paths tested;
- end-to-end scenario results;
- security findings and fixes;
- backup/restore evidence;
- remaining optional enhancements separated from MVP gaps;
- final release recommendation: ready, conditionally ready, or not ready, with reasons.

Standard Phase Completion Report

Require the agent to use this report after every phase:

Phase completed:

- Phase number and title

Files changed:

- ...

Requirements completed:

- MV-FR-...

Features implemented:

- ...

Security controls verified:

- ...

Commands run:

- ...

Automated test results:

- ...

Manual/browser verification:

- ...

Migration and data reconciliation:

- ...

Known gaps or follow-ups:

- ...

Next recommended phase:

- ...

Mandatory Stop Conditions

Do not start the next phase when any of these is true:

- A Cafe user can retrieve any Retail confidential record or identifying metadata.
- A Retail user can retrieve Cafe data outside authorization.
- An endpoint trusts client-provided company, role, branch, price, discount, tax, or payment state.
- A QR guest can access another table session or alter a server-calculated price.
- Duplicate order or bill requests create duplicate records, stock movements, payments, or revenue.
- Non-GST mode applies/displays GST or GSTIN.
- Existing Retail stock, invoice, payment, ledger, dashboard, export, or AI regression fails.
- A migration creates orphaned or company-mismatched rows.
- A destructive action runs without scope, reason, audit, backup, dependency, period, and approval controls.
- The required backup cannot be restored.

Final Execution Advice

- Keep one task/thread per phase so logs and reports remain reviewable.
- Save every phase report in `docs/phase-reports/` before moving on.
- Re-run the Phase 2 cross-venture suite after every later phase that adds an API, export, AI tool, or dashboard.
- Re-run invoice, stock, payment, and ledger regression after every later phase that changes ordering or billing.
- Use PostgreSQL, not SQLite alone, for final migration, lock, uniqueness, and concurrency evidence.
- Keep remote public testing disabled until Phase 11 hardening is complete.